

CO3 - AT1- Analytical Problem solving[← Back](#)

QUESTIONS

Q1CO3 - AT1 - Analytical Problem Solving
20 marks**Q2**CO3 - AT1 - Analytical Problem Solving
20 marks**Q3**CO3 - AT1 - Analytical Problem solving
20 marks**Q4**CO3 - AT1 - Analytical Problem Solving
20 marks**Q5**CO3 - AT1 - Analytical Problem Solving
20 marks

5/5 answered

Q2 CO3 - AT1 - Analytical Problem Solving

20 marks

2. Given an array of integers, write a C program that:
- i. Determines whether the array is sorted.
 - ii. If sorted, performs Binary Search.
 - iii. Otherwise, performs Linear Search.
 - iv. Displays the search method used and the number of comparisons performed.

Your Code

```
1 #include <stdio.h>
2
3 int main() {
4     int n, i, key;
5     int a[100];
6     int sorted = 1;
7     int comparisons = 0;
8
9     printf("Enter number of elements: ");
10    scanf("%d", &n);
11
12    printf("Enter array elements:\n");
13    for(i = 0; i < n; i++) {
14        scanf("%d", &a[i]);
15    }
```

Input

```
5
1 3 5 7 9
7
```

Output

CO3 - AT1- Analytical Problem solving

[← Back](#)

QUESTIONS

Q1

CO3 - AT1 - Analytical Problem Solving
20 marks

Q2

CO3 - AT1 - Analytical Problem Solving
20 marks

Q3

CO3 - AT1 - Analytical Problem solving
20 marks

Q4

CO3 - AT1 - Analytical Problem Solving
20 marks

Q5

CO3 - AT1 - Analytical Problem Solving
20 marks

5/5 answered

Q3 CO3 - AT1 - Analytical Problem solving

20 marks

3. Store marks of N students in M subjects using a two-dimensional array. Generate a report containing:

- Student topper
- Subject topper
- Class average
- Subject-wise average
- Students scoring above the overall average
- Rank list without modifying the original data

Your Code

```
1 #include <stdio.h>
2
3 int main() {
4     int n, m, i, j;
5
6     printf("Enter number of students: ");
7     scanf("%d", &n);
8
9     printf("Enter number of subjects: ");
10    scanf("%d", &m);
11
12    int marks[n][m];
13    int total[n], subject total[m];
```

Input

```
3
3
90 80 70
85 95 75
```

Output



CO3 - AT1- Analytical Problem solving[← Back](#)**QUESTIONS****Q1**CO3 - AT1 - Analytical Problem Solving
20 marks**Q2**CO3 - AT1 - Analytical Problem Solving
20 marks**Q3**CO3 - AT1 - Analytical Problem solving
20 marks**Q4**CO3 - AT1 - Analytical Problem Solving
20 marks**Q5**CO3 - AT1 - Analytical Problem Solving
20 marks

5/5 answered

Q4 CO3 - AT1 - Analytical Problem Solving

20 marks

4. Given N strings, write a C program to:
- a) Sort them lexicographically.
 - b) Find the longest and shortest strings.
 - c) Count duplicate strings.
 - d) Display strings beginning with vowels.

Your Code

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <ctype.h>
4
5 int main() {
6     int n, i, j;
7     char str[100][100], temp[100];
8
9     scanf("%d", &n);
10
11     for(i = 0; i < n; i++) {
12         scanf("%s", str[i]);
13     }
14
15     // a) Sorting
```

Input

```
6
apple
banana
orange
```

Output

CO3 - AT1- Analytical Problem solving[← Back](#)**QUESTIONS****Q1**CO3 - AT1 - Analytical Problem
Solving
20 marks**Q2**CO3 - AT1 - Analytical Problem
Solving
20 marks**Q3**CO3 - AT1 - Analytical Problem
solving
20 marks**Q4**CO3 - AT1 - Analytical Problem
Solving
20 marks**Q5**CO3 - AT1 - Analytical Problem
Solving
20 marks

5/5 answered

Q5 CO3 - AT1 - Analytical Problem Solving

20 marks

5. Accept a string containing repeated characters. Compress it by replacing consecutive repeated characters with the character followed by its count.

Example:

Input:

aaabbbcccd

Output:

a3b3c4d2

Also calculate the compression ratio.

Your Code

```
1 #include <stdio.h>
2
3 int main() {
4     char str[1000], result[1000];
5     int i, j = 0, count;
6
7     scanf("%s", str);
8
9     int n = strlen(str);
10
11     for(i = 0; i < n; i++) {
12         count = 1;
```

Input

aaabbbcccd

Output

CO3 - AT1- Analytical Problem solving

[← Back](#)

QUESTIONS

Q1 ✓
CO3 - AT1 - Analytical Problem Solving
20 marks

Q2 ✓
CO3 - AT1 - Analytical Problem Solving
20 marks

Q3 ✓
CO3 - AT1 - Analytical Problem solving
20 marks

Q4 ✓
CO3 - AT1 - Analytical Problem Solving
20 marks

Q5 ✓
CO3 - AT1 - Analytical Problem Solving
20 marks

5/5 answered

Q1 CO3 - AT1 - Analytical Problem Solving

20 marks

1. Given an $N \times N$ matrix, Write a C Program to determine whether it is:
- a) Symmetric
 - b) Upper triangular
 - c) Lower triangular
 - d) Diagonal matrix
 - e) Identity matrix
 - f) Display all applicable classifications.

Your Code

```
1 #include <stdio.h>
2
3 int main() {
4     int n, i, j;
5     int a[10][10];
6
7     int symmetric = 1, upper = 1, lower = 1, diagonal = 1, identity = 1;
8
9     printf("Enter order of matrix (N x N): ");
10    scanf("%d", &n);
11
12    printf("Enter matrix elements:\n");
13    for(i = 0; i < n; i++) {
```

Input

```
1 0 0
0 1 0
0 0 1
```

Output